SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Deep Learning **COURSE CODE:** MLA0407

COURSE OUTCOME COVERED

CO1: Apply the fundamental concepts of machine learning and neural networks to design and analyze learning algorithms using perceptrons, multilayer perceptrons, forward propagation, backpropagation, gradient descent optimization techniques, and Maximum Likelihood Estimation for solving real-world problems. (BL3)

Duration: 1 Hour

QUESTIONS: 10 **Total Marks:50**

Annexure A

Experiments

1. Implement a Single Artificial Neuron using Python.
2. Implement the Perceptron Learning Algorithm for the AND Logic Gate.
3. Design and Implement a Perceptron Model for the OR Logic Gate.
4. Study the Limitations of a Single-Layer Perceptron using XOR Classification.
5. Implement the Gradient Descent Optimization Algorithm.
6. Compare Batch Gradient Descent, Stochastic Gradient Descent (SGD), and Mini-Batch Gradient Descent.
7. Design and Train a Multilayer Perceptron (MLP) for Classification.
8. Implement Forward Propagation for a Neural Network with One Hidden Layer.
9. Implement the Backpropagation Algorithm for a Neural Network.
10. Analyze the Curse of Dimensionality in Machine Learning.

Code of Conduct:

*I **P. DEEKSHANA / 192425104** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO1 – AT3 – EXPERIMENTS

NAME: P. DEEKSHANA

REG NO: 192425104

COURSE CODE: MLA0407

COURSE NAME: DEEP LEARNING

Experiment 1: Single Artificial Neuron

1.py - C:/Users/DEEKSHANA PRABHU/Downloads/1.py (3.9.13)

File Edit Format Run Options Window Help

```
import numpy as np

def step(x):
    return 1 if x >= 0 else 0

weights = np.array([0.5, 0.3, -0.2])
bias = -0.4

inputs = [
    [1, 1, 1],
    [1, 0, 1],
    [0, 1, 0],
    [1, 1, 0]
]

for x in inputs:
    net = np.dot(weights, x) + bias
    output = step(net)
    print("Input:", x, "Net:", net, "Output:", output)
```

IDLE Shell 3.9.13

File Edit Shell Debug Options Window Help

```
Python 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/DEEKSHANA PRABHU/Downloads/1.py ==========
Input: [1, 1, 1] Net: 0.20000000000000007 Output: 1
Input: [1, 0, 1] Net: -0.10000000000000003 Output: 0
Input: [0, 1, 0] Net: -0.10000000000000003 Output: 0
Input: [1, 1, 0] Net: 0.4 Output: 1
>>>
```

Experiment 2: Perceptron for AND Gate

1.py - C:/Users/DEEKSHANA PRABHU/Downloads/1.py (3.9.13)

File Edit Format Run Options Window Help

```
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0,0,0,1])

weights = np.zeros(2)
bias = 0
lr = 0.1

for epoch in range(10):
    for i in range(len(X)):
        net = np.dot(X[i], weights) + bias
        pred = 1 if net >= 0 else 0
        error = y[i] - pred
        weights += lr * error * X[i]
        bias += lr * error

print("Weights:", weights)
print("Bias:", bias)

for x in X:
    pred = 1 if np.dot(x, weights)+bias >= 0 else 0
    print(x, "->", pred)
```

IDLE Shell 3.9.13

File Edit Shell Debug Options Window Help

```
Python 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/DEEKSHANA PRABHU/Downloads/1.py ==========
Weights: [0.2 0.1]
Bias: -0.20000000000000004
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1
>>>
```

Experiment 3: Perceptron for OR Gate

1.py - C:/Users/DEEKSHANA PRABHU/Downloads/1.py (3.9.13)

File Edit Format Run Options Window Help

```
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0,1,1,1])

weights = np.zeros(2)
bias = 0
lr = 0.1

for epoch in range(10):
    for i in range(len(X)):
        net = np.dot(X[i], weights) + bias
        pred = 1 if net >= 0 else 0
        error = y[i] - pred
        weights += lr * error * X[i]
        bias += lr * error

print("Weights:", weights)
print("Bias:", bias)

for x in X:
    pred = 1 if np.dot(x, weights)+bias >= 0 else 0
    print(x, "->", pred)
```

IDLE Shell 3.9.13

File Edit Shell Debug Options Window Help

```
Python 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/DEEKSHANA PRABHU/Downloads/1.py ==========
Weights: [0.1 0.1]
Bias: -0.1
[0 0] -> 0
[0 1] -> 1
[1 0] -> 1
[1 1] -> 1
>>>
```

Experiment 4: XOR Classification

1.py - C:/Users/DEEKSHANA PRABHU/Downloads/1.py (3.9.13)

File Edit Format Run Options Window Help

```
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0,1,1,0])

weights = np.zeros(2)
bias = 0
lr = 0.1

for epoch in range(20):
    for i in range(len(X)):
        net = np.dot(X[i], weights) + bias
        pred = 1 if net >= 0 else 0
        error = y[i] - pred
        weights += lr * error * X[i]
        bias += lr * error

print("Final Weights:", weights)
print("Bias:", bias)

for x in X:
    pred = 1 if np.dot(x, weights)+bias >= 0 else 0
    print(x, "->", pred)

print("Single-layer perceptron cannot correctly classify XOR.")
```

IDLE Shell 3.9.13

File Edit Shell Debug Options Window Help

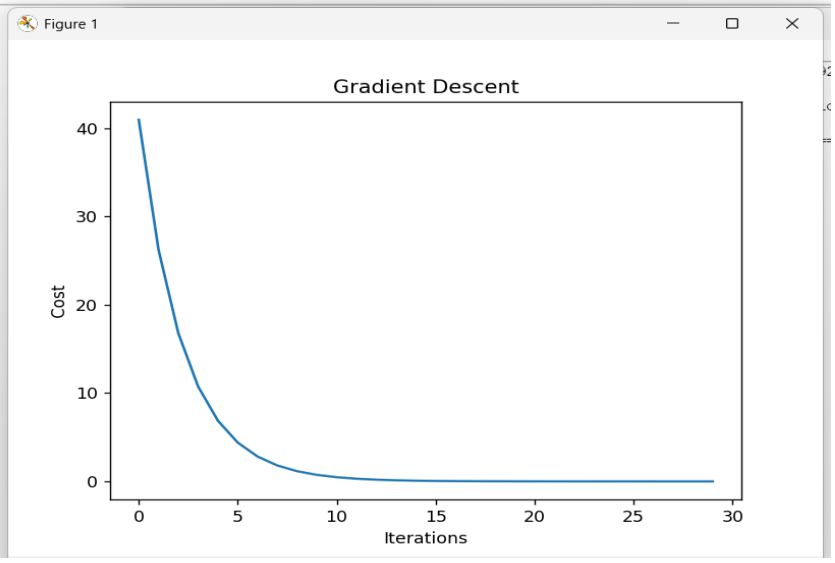
```
Python 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/DEEKSHANA PRABHU/Downloads/1.py ==========
Final Weights: [-0.1  0. ]
Bias: 0.0
[0 0] -> 1
[0 1] -> 1
[1 0] -> 0
[1 1] -> 0
Single-layer perceptron cannot correctly classify XOR.
>>>
```

Experiment 5: Gradient Descent

1.py - C:/Users/DEEKSHANA PRABHU/Downloads/1.py (3.9.13)

File Edit Format Run Options Window Help

```
import matplotlib.pyplot as plt

x = 8
lr = 0.1
iterations = 30

costs = []

for i in range(iterations):
    grad = 2 * x
    x = x - lr * grad
    cost = x**2
    costs.append(cost)

plt.plot(costs)
plt.xlabel("Iterations")
plt.ylabel("Cost")
plt.title("Gradient Descent")
plt.show()
```



Experiment 6: Batch GD, SGD, Mini-Batch GD

1.py - C:/Users/DEEKSHANA PRABHU/Downloads/1.py (3.9.13)

File Edit Format Run Options Window Help

```
from sklearn.linear_model import SGDRegressor, LinearRegression
from sklearn.datasets import make_regression
import numpy as np

X, y = make_regression(n_samples=1000, n_features=5, noise=10, random_state=42)

batch = LinearRegression()
batch.fit(X, y)

sgd = SGDRegressor(max_iter=1000)
sgd.fit(X, y)

print("Batch Coefficients:")
print(batch.coef_)

print("\nSGD Coefficients:")
print(sgd.coef_)

mini_batch_size = 100
weights = np.zeros(X.shape[1])

lr = 0.001

for epoch in range(20):
    for i in range(0, len(X), mini_batch_size):
        xb = X[i:i+mini_batch_size]
        yb = y[i:i+mini_batch_size]

        pred = xb.dot(weights)
        grad = xb.T.dot(pred - yb)/len(xb)
        weights -= lr * grad

print("\nMini-Batch Weights:")
print(weights)
```

IDLE Shell 3.9.13

File Edit Shell Debug Options Window Help

```
Python 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/DEEKSHANA PRABHU/Downloads/1.py ==========
Batch Coefficients:
[28.05749028 45.94154566 16.61187941 24.53193249 19.36893323]

SGD Coefficients:
[28.07062898 45.98449664 16.50381414 24.56760681 19.44934155]

Mini-Batch Weights:
[4.54951106 8.52797805 3.40216796 4.35993058 3.50358266]
>>>
```

Experiment 7: Multilayer Perceptron (MLP)

1.py - C:/Users/DEEKSHANA PRABHU/Downloads/1.py (3.9.13)

File Edit Format Run Options Window Help

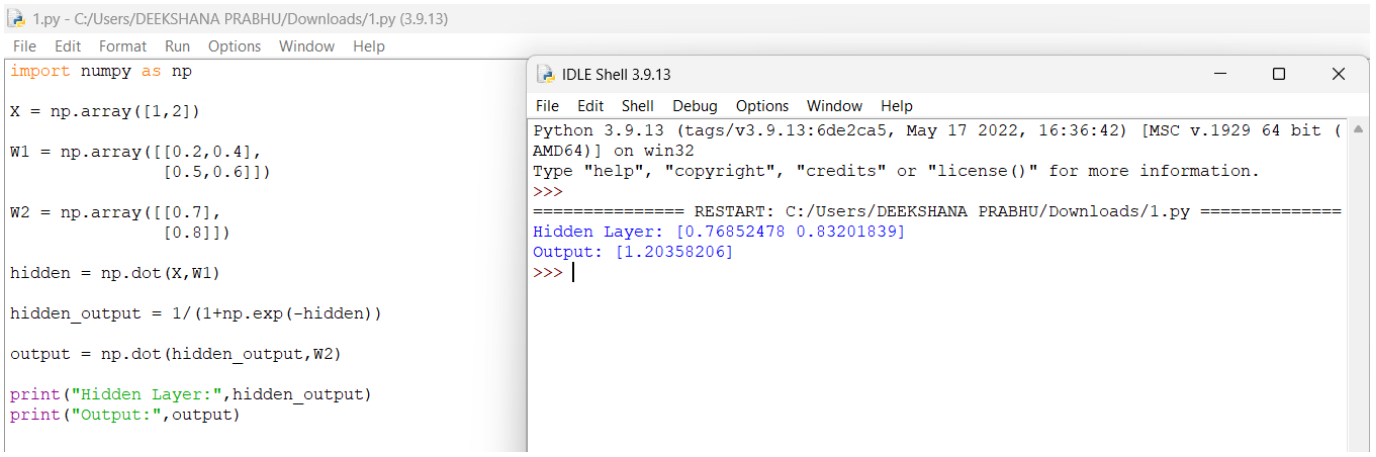
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create MLP model
model = MLPClassifier(hidden_layer_sizes=(10,), max_iter=500, random_state=42)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

IDLE Shell 3.9.13

File Edit Shell Debug Options Window Help

```
Python 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/DEEKSHANA PRABHU/Downloads/1.py ==========

Warning (from warnings module):
  File "C:\Users\DEEKSHANA PRABHU\AppData\Local\Programs\Python\Python39\lib\sit
e-packages\sklearn\neural_network\_multilayer_perceptron.py", line 691
    warnings.warn(
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (500) reached and t
he optimization hasn't converged yet.
Accuracy: 0.9333333333333333

Confusion Matrix:
[[10  0  0]
 [ 0  7  2]
 [ 0  0 11]]
>>>
```

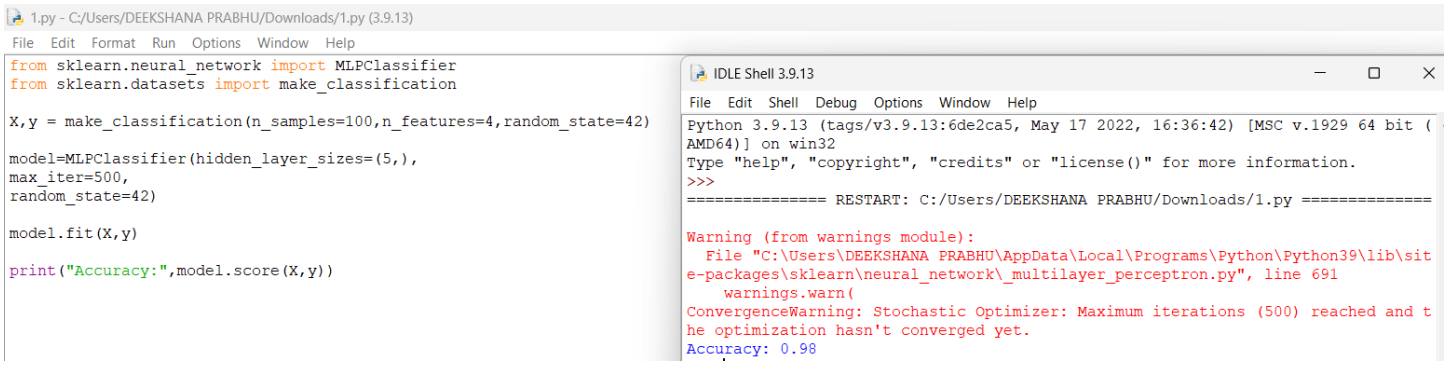
Experiment 8: Forward Propagation



```
1.py - C:/Users/DEEKSHANA PRABHU/Downloads/1.py (3.9.13)
File  Edit  Format  Run  Options  Window  Help
import numpy as np

X = np.array([1,2])

W1 = np.array([[0.2,0.4],
               [0.5,0.6]])

W2 = np.array([[0.7],
               [0.8]])

hidden = np.dot(X,W1)

hidden_output = 1/(1+np.exp(-hidden))

output = np.dot(hidden_output,W2)

print("Hidden Layer:",hidden_output)
print("Output:",output)
```

```
IDLE Shell 3.9.13
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/DEEKSHANA PRABHU/Downloads/1.py ==========
Hidden Layer: [0.76852478 0.83201839]
Output: [1.20358206]
>>>
```

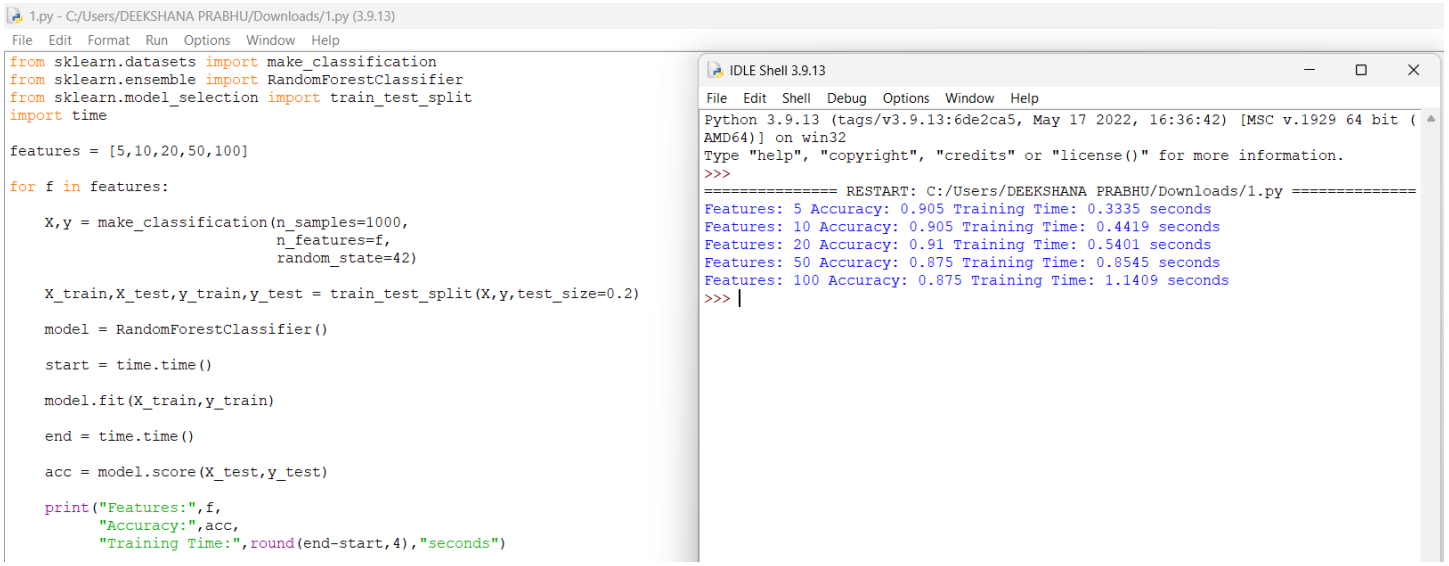
Experiment 9: Backpropagation



```
1.py - C:/Users/DEEKSHANA PRABHU/Downloads/1.py (3.9.13)
File  Edit  Format  Run  Options  Window  Help
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification

X,y = make_classification(n_samples=100,n_features=4,random_state=42)

model=MLPClassifier(hidden_layer_sizes=(5,),
max_iter=500,
random_state=42)

model.fit(X,y)

print("Accuracy:",model.score(X,y))
```

```
IDLE Shell 3.9.13
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/DEEKSHANA PRABHU/Downloads/1.py ==========

Warning (from warnings module):
  File "C:\Users\DEEKSHANA PRABHU\AppData\Local\Programs\Python\Python39\lib\sit
e-packages\sklearn\neural_network\_multilayer_perceptron.py", line 691
    warnings.warn(
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (500) reached and t
he optimization hasn't converged yet.
Accuracy: 0.98
```

Experiment 10: Curse of Dimensionality



```
1.py - C:/Users/DEEKSHANA PRABHU/Downloads/1.py (3.9.13)
File  Edit  Format  Run  Options  Window  Help
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
import time

features = [5,10,20,50,100]

for f in features:

    X,y = make_classification(n_samples=1000,
                              n_features=f,
                              random_state=42)

    X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2)

    model = RandomForestClassifier()

    start = time.time()

    model.fit(X_train,y_train)

    end = time.time()

    acc = model.score(X_test,y_test)

    print("Features:",f,
          "Accuracy:",acc,
          "Training Time:",round(end-start,4),"seconds")
```

```
IDLE Shell 3.9.13
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/DEEKSHANA PRABHU/Downloads/1.py ==========
Features: 5 Accuracy: 0.905 Training Time: 0.3335 seconds
Features: 10 Accuracy: 0.905 Training Time: 0.4419 seconds
Features: 20 Accuracy: 0.91 Training Time: 0.5401 seconds
Features: 50 Accuracy: 0.875 Training Time: 0.8545 seconds
Features: 100 Accuracy: 0.875 Training Time: 1.1409 seconds
>>>
```

GITHUB LINK: https://github.com/Deekshana-Prabhu/C01-AT3-Experiments.git

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Content Accuracy	30	Highly accurate, indepth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____